JavaScript

The Language of the Web

Computer Science — 1st Year Presentation

Group Members

Boualem Bayoud | Anis Kara

Abdesselam Ahmed Amine | Abd El Djalil Adjou

University of Bouira | April 5, 2026

JavaScript is a `high-level`, `interpreted` and `dynamic` programming language — the **only** language that runs natively inside every web browser.

Front-End

Makes web pages **interactive**, animated and responsive — without reloading from the server.

Back-End

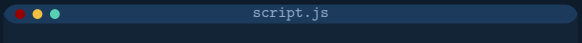
Runs on servers via **Node.js** to handle databases, APIs and real-time applications.

Mobile & Desktop

Cross-platform apps via **React Native** (mobile) and **Electron** (desktop).

Key Facts

- Used by **98%** of all websites worldwide
- Runs in every modern browser — *no installation needed*
- Part of the essential **HTML / CSS / JS** trio
- **#1** most used language on GitHub for years
- Huge ecosystem: **React, Vue, Angular, Node.js**



History of JavaScript › From 10 Days of Work to the World's Most Used Language

1995 — **Brendan Eich** creates **Mocha** (later renamed **JavaScript**) at *Netscape* in just **10 days**.

1996 — Microsoft releases JScript for Internet Explorer — the "browser wars" begin.

1997 — Standardised as **ECMAScript** by ECMA International — one specification for all browsers.

2009 — **Node.js** released — JavaScript now runs on *servers*, not just browsers.

Now — **#1** language on GitHub. Powering the entire modern web.

Creator of JavaScript

Brendan Eich

- Born: July 4, 1961
- American programmer
- Co-founder of Mozilla
- Created JavaScript in **May 1995**
- Built in only **10 days!**

Why was JavaScript created?

Netscape needed a **lightweight scripting language** so that web designers — not just programmers — could make pages **interactive** without reloading them from the server every time.

CHAPTER 2

Control & Logic

Conditions ▪ Loops ▪ Functions ▪ Frameworks

Abdesselam Ahmed Amine

Introduction to Artificial Intelligence

Chapter Overview › What we will cover in the next ~6 minutes

Conditions

Deciding what code runs

Loops

Repeating actions

Functions

Reusable blocks of code

Frameworks

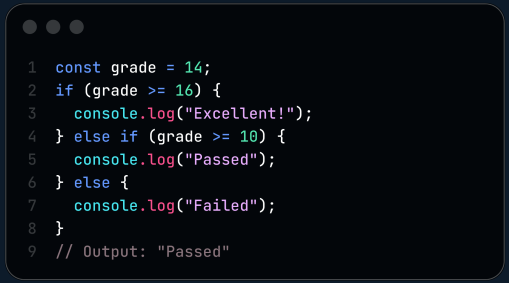
React & Node.js

Why does this matter?

Control structures are the **skeleton** of every program. Without them, code would just run line-by-line with no decisions, no repetition, and no reuse.

- Code runs **only when** a condition is `true`
- `else if` chains multiple conditions
- `else` is the final fallback
- Conditions use **comparison operators**:

  `===` `!==` `>` `<` `>=` `<=`



```
1  const grade = 14;
2  if (grade >= 16) {
3    console.log("Excellent!");
4  } else if (grade >= 10) {
5    console.log("Passed");
6  } else {
7    console.log("Failed");
8  }
9  // Output: "Passed"
```

Remember

Always use `===` (strict equality), not `==` (loose equality).

Conditions — Ternary & Switch › Shorter syntax for simple choices

Ternary Operator

```
// condition ? valueA : valueB
const age = 20;
const status = age >= 18
  ? "Adult"
  : "Minor";
console.log(status); // "Adult"
```

Best for **simple, one-line** decisions.
Avoid nesting ternaries — hard to read.

Switch Statement

```
const day = "Monday";
switch (day) {
  case "Monday":
    console.log("Week starts!");
    break;
  case "Friday":
    console.log("Weekend soon!");
    break;
  default:
    console.log("Regular day");
}
```

`break` stops fall-through.

A `for` loop has **3 parts**:

- **Initializer** — `let i = 0`
  Runs once at the start
- **Condition** — `i < 5`
  Checked before each iteration
- **Update** — `i++`
  Runs after each iteration

```
// Count from 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// Loop over an array
const students = [
  "Ali", "Sara", "Youssef"
];
for (let i = 0;
     i < students.length; i++) {
  console.log(students[i]);
}
```

Output

```
1 2 3 4 5
```

Loops — `while` & `for...of` › Two more ways to repeat code

while — Unknown iterations

```
let count = 0;

while (count < 3) {
  console.log("Count: " + count);
  count++;
}
// Count: 0
// Count: 1
// Count: 2
```

⚠ Always update the variable inside the loop or it runs forever!

for...of — Clean array iteration

```
const fruits = [
  "apple", "mango", "fig"
];

for (const fruit of fruits) {
  console.log(fruit);
}
// apple
// mango
// fig
```

✓ Cleaner than `for` when you don't need the index. Preferred modern JS.

Functions — Reusable Blocks of Code › Write once, use anywhere

- A function is a **named block** of code
- It runs only when **called**
- It can receive **parameters** (inputs)
- It can **return** a result

Two declaration styles

Declaration: can be used before it is defined (hoisted)

Expression: stored in a variable

```
// Function Declaration
function greet(name) {
  return "Hello, " + name + "!";
}
console.log(greet("Ahmed"));
// Hello, Ahmed!
// Function Expression
const square = function(n) {
  return n * n;
};
console.log(square(5)); // 25
```

Functions — Arrow Functions (ES6) › A shorter, modern syntax introduced in 2015

- Introduced in **ES6 (2015)**
- Shorter syntax with `=>`
- No `function` keyword needed
- One-liners can skip `return` & braces
- Very common in modern codebases

Rule of thumb

Use arrow functions for short, simple tasks.
Use regular functions for complex logic.

```
// Regular function
function add(a, b) {
  return a + b;
}
// Same — arrow style
const add = (a, b) => a + b;
console.log(add(3, 4)); // 7
// Multi-line arrow function
const describe = (name, age) => {
  const msg =
    `${name} is ${age} years old.`;
  return msg;
};
console.log(describe("Sara", 20));
// Sara is 20 years old.
```

What is a Framework?

A framework is a **collection of pre-written code** that gives you a structure to build applications faster, without starting from zero.

React

Runs in the **browser**
(Front-end / Client-side)

Built by Meta (Facebook)
Used for building user interfaces

Component-based architecture

vs

Node.js

Runs on the **server**
(Back-end / Server-side)

Built on Chrome's V8 engine
Used for APIs & web servers

Non-blocking, event-driven

- Created by **Meta** in 2013
- Builds UIs from small reusable pieces called **components**
- Uses **JSX** — HTML-like syntax inside JavaScript
- Updates only the parts of the page that changed (**Virtual DOM**)

Used by

Facebook, Instagram, Airbnb, Netflix, WhatsApp Web

```jsx
// A simple React component
function WelcomeCard({ name }) {
  return (
    <div className="card">
      <h2>Hello, {name}!</h2>
      <p>Welcome to our app.</p>
    </div>
  );
}
// Using the component
function App() {
  return <WelcomeCard name="Ahmed" />;
}
```

JSX looks like HTML but it is JavaScript under the hood.

- Created by **Ryan Dahl** in 2009
- Lets JS run on a **server**, not just a browser
- Powers REST APIs, chat apps, real-time tools
- Uses **npm** — the world's largest package registry
- Non-blocking: handles many users **simultaneously**

Used by

LinkedIn, Netflix, Uber, PayPal, NASA

```
// A simple web server with Node.js
const http = require("http");
const server = http.createServer(
  (request, response) => {
    response.writeHead(200, {
      "Content-Type": "text/plain"
    });
    response.end(
      "Hello from Node.js!"
    );
  }
);
server.listen(3000, () => {
  console.log(
    "Server running on port 3000"
  );
});
```

Chapter 2 — Summary › Key takeaways

Conditions (`if` / `else` , `switch`)
Let your program make decisions

Loops (`for` , `while` , `for...of`)
Repeat code efficiently

Functions (declaration, expression, arrow)
Write once, reuse everywhere

Frameworks (React, Node.js)
Build real applications faster

→ What comes next

Chapter 3 will cover:

- Interacting with HTML (DOM)
- Handling events
- Common uses of JavaScript
- Advantages & limitations